Problema Atual

Contratos de mútuo entre empresas são instrumentos financeiros amplamente utilizados em grupos empresariais para transferência interna de recursos, financiamento de capital de giro e otimização da gestão de caixa. Nesses contratos, uma empresa do grupo concede empréstimo a outra entidade relacionada, devendo a operação respeitar condições de mercado, incidência tributária adequada e requisitos formais de registro.

No Brasil, a legislação estabelece que operações de mútuo intercompany devem observar parâmetros de mercado e obrigações fiscais específicas, com o objetivo de evitar distorções contábeis, planejamento tributário abusivo e transferência indevida de resultados entre empresas relacionadas. A ausência de controles adequados nessas operações pode expor o grupo empresarial a riscos relevantes de compliance, incluindo autuações fiscais, penalidades regulatórias e questionamentos por parte de órgãos de fiscalização.

Além do risco regulatório, a concessão de crédito interno sem **avaliação estruturada** pode resultar em **inadimplência**, renegociação compulsória ou deterioração do fluxo de caixa consolidado do grupo. Essas ocorrências podem gerar impactos financeiros significativos, como perdas diretas, aumento do custo de capital e restrições operacionais decorrentes de sanções legais ou desequilíbrios financeiros internos.

Porque usar machine learning ?

A aplicação de machine learning será integrada a um sistema financeiro corporativo já estruturado, que consolida informações operacionais e contábeis relevantes para análise de risco. O sistema registra e organiza dados provenientes de múltiplas fontes, incluindo fluxo de caixa, contas correntes, aplicações financeiras, captações, contratos de mútuo, derivativos, contas a pagar e contas a receber.

Embora essas informações estejam disponíveis, a análise manual — frequentemente realizada por meio de planilhas — exige a consolidação de dados heterogêneos, provenientes de diferentes módulos operacionais. Esse processo é complexo, sujeito a inconsistências e altamente dependente de interpretação humana, o que aumenta a probabilidade de erros e dificulta a identificação de padrões multivariados relevantes.

Métodos tradicionais baseados em regras fixas ou análises isoladas de indicadores financeiros não capturam adequadamente interações não lineares entre variáveis, nem permitem a avaliação sistemática de múltiplos fatores de risco de forma simultânea. Nesse contexto, técnicas de machine learning supervisionado tornam-se adequadas por possibilitarem a modelagem de relações complexas entre variáveis financeiras, automatizando a detecção de padrões históricos associados à inadimplência e apoiando decisões mais consistentes e escaláveis.

Principais Desafios do Domínio

A avaliação de risco em contratos de mútuo entre empresas envolve múltiplos desafios técnicos. O risco financeiro resulta da interação simultânea entre diversos indicadores econômicos e operacionais, frequentemente com relações não lineares e difíceis de modelar por regras fixas.

Os dados financeiros apresentam ruído decorrente de variações contábeis, sazonalidade e diferenças operacionais, o que reduz a previsibilidade direta dos indicadores. Além disso, eventos de inadimplência tendem a ser menos frequentes, gerando desbalanceamento de classes e exigindo cuidado na avaliação dos modelos.

Outro fator relevante é o custo assimétrico dos erros de classificação, pois falhas na identificação de contratos de alto risco podem gerar perdas financeiras significativas. Soma-se a isso a necessidade de interpretabilidade das decisões e a integração de dados heterogêneos provenientes de diferentes módulos financeiros.

Definição Formal da Variável-Alvo

A variável-alvo do modelo representa o **status de adimplência do contrato de mútuo durante o horizonte de monitoramento definido**.

Formalmente, define-se:

1: Se ocorreu inadimplência relevante no período de observação;
0: Caso contrário;

Considera-se inadimplência relevante qualquer ocorrência de:

- atraso superior a um limite operacional predefinido (por exemplo, 60 dias), ou
- necessidade de renegociação contratual associada a incapacidade de pagamento, ou
- não liquidação integral do contrato no prazo previsto.

Assim, a variável-alvo é binária e indica a ocorrência ou não de deterioração significativa do cumprimento financeiro do contrato.

Essa definição está alinhada com práticas de gestão de risco de crédito, nas quais eventos de default são operacionalizados como indicadores objetivos de falha no cumprimento das obrigações financeiras.

Descrição Técnica do Dataset

O dataset utilizado representa contratos de mútuo intercompany, com observações individuais correspondendo a contratos financeiros concedidos entre entidades relacionadas.

Estrutura geral

- Cada linha representa um contrato de mútuo.
- Cada coluna representa um atributo financeiro, operacional ou de risco da entidade tomadora ou da estrutura do contrato.
- O conjunto de dados contém múltiplas variáveis contínuas e discretas de natureza econômico-financeira.

Volume de dados

O conjunto contém 10 mil observações, permitindo análise estatística multivariada consistente e viabilizando a aplicação de modelos supervisionados com capacidade de generalização.

Natureza das variáveis explicativas

As features descrevem diferentes dimensões do risco financeiro, incluindo:

- **Valor Nominal (valor_contrato)**

  Corresponde ao valor total do contrato de mútuo firmado entre as partes.

  Tipo: Numérico contínuo (float)

  Escala: Valor monetário (R$)

- **Prazo (prazo_meses)**

  Representa a duração do contrato até o vencimento final.

  Tipo: Numérico Inteiro

  Escala: Meses

- **Taxa de Juros (taxa_juros)**

Percentual de juros acordado no contrato.

Tipo: Numérico contínuo

Escala: Percentual em formato decimal

- **Margem Operacional (ebitda_margem)**

  Indica a eficiência operacional da empresa, medindo o lucro operacional em relação à receita.

  Tipo: Numérico contínuo

  Escala: Percentual em formato decimal (pode ser negativo)

- **Liquidez (indice_liquidez_corrente)**

  Capacidade da empresa de cumprir obrigações de curto prazo.

  Tipo: Numérico contínuo

  Escala: Razão financeira (ex: 1.5 significa que possui 1.5x ativos sobre passivos no curto prazo)

- **Alavancagem (indice_endividamento)**

  Nível de endividamento da empresa em relação ao patrimônio ou EBITDA.

  Tipo: Numérico contínuo

  Escala: Razão financeira (0 a 1) - Valores próximos a 1 indicam alto endividamento

- **Capacidade de Pagamento de Juros (cobertura_juros)**

  Mede quantas vezes o resultado operacional cobre as despesas financeiras.

  Tipo: Numérico contínuo

  Escala: Razão (ex: 2.0 = cobre duas vezes os juros)

- **Volatilidade de Resultados (volatilidade_receira)**

  Mede a variação dos resultados operacionais ao longo do tempo (ex: desvio padrão da margem ou lucro).

  Tipo: Numérico contínuo

  Escala: Índice Percentual - Valores maiores indicam maior instabilidade

- **Crescimento de Receita (crescimento_receita_12m)**

  Variação percentual da receita no período de 12 meses.

  Tipo: Numérico contínuo

  Escala: Percentual (pode ser negativo)

- **Ciclo de caixa (ciclo_caixa)**

  Número médio de dias necessários para converter operações em caixa.

  Tipo: Numérico contínuo

  Escala: Dias - Valores maiores indicam maior necessidade de capital de giro.

- **Histórico de Atrasos (histórico_atrasos_12m)**

  Indica ocorrências de atraso em pagamentos anteriores.

  Tipo: Numérico discreto (quantidade de atrasos)

- **Exposição Cambial (exposicao_cambial)**

  Percentual da receita ou dívida atrelada à moeda estrangeira.

  Tipo: Numérico contínuo

  Escala: Percentual 0 a 1 (0.25 = 25% exposto ao câmbio)

- **Variação do Capital de Giro (variacao_capital_giro)**

  Mede alterações no capital de giro da empresa ao longo do tempo.

  Tipo: Numérico contínuo

  Escala: Percentual em formato decimal (Pode ser positivo ou negativo)

- **Score de Risco (rating_interno_score)**

  Nota atribuída internamente para indicar probabilidade de inadimplência.

  Tipo: Numérico contínuo

  Escala: Normalizado entre 0 e 1 - Quanto maior, maior a qualidade de crédito

- **Cobertura por Garantias (garantia_ratio)**

  Percentual do valor do contrato coberto por garantias reais ou fidejussórias.

Tipo: Numérico contínuo

Escala: Percentual (0 até valores acima de 1)

- **default (Variável Alvo)**

  Indica ocorrência de inadimplência.

  Tipo: Numérico binário

  Escala: 0 = Não inadimplente, 1 = Inadimplente

Tipo de problema supervisionado

A presença de rótulos históricos de inadimplência permite treinamento supervisionado, no qual o modelo aprende padrões associados a eventos passados para prever novos casos.

Justificativa da Classificação Binária

O problema é formulado como classificação binária porque a decisão operacional associada à análise de risco é essencialmente dicotômica: o contrato é considerado aceitável sob condições padrão ou apresenta risco elevado que exige intervenção.

Embora o risco financeiro seja um fenômeno contínuo, a tomada de decisão corporativa normalmente requer categorização objetiva para fins de:

- concessão ou restrição de crédito interno
- definição de garantias adicionais
- ajuste de taxa de juros
- monitoramento reforçado
- reprovação da operação

A transformação do risco em duas classes — inadimplente ou não inadimplente — permite alinhar diretamente a saída do modelo com ações operacionais concretas.

Além disso, a classificação binária:

- simplifica a interpretação do modelo

- facilita a definição de métricas de desempenho
- permite análise clara de trade-offs entre tipos de erro
- é compatível com práticas tradicionais de modelagem de risco de crédito

Portanto, a formulação binária não apenas reflete a estrutura das decisões financeiras reais, mas também fornece um enquadramento estatístico adequado para avaliação e comparação de modelos supervisionados.

Modelo Baseline: Classificador Linear com Perceptron

Segue abaixo exemplos típicos de combinações que possuem maior incidência de erros de classificação de acordo com a regra de negócio do problema.

- Empresas com altas dívidas mas alta geração de caixa.
- Empresas com baixa liquidez mas fortes garantias.

O modelo foi criado com os seguintes parâmetros:

test_size=0.3 - 30% dos dados foram separados para teste do modelo.

random_state=12 - 12 porque é a quantidade de títulos do campeonato brasileiro do Palmeiras.

stratify=y - Serve para preservar a proporção original das classes. Logo, se os dados são 20% inadimplentes, ao separar treino e teste irá manter em ambos os conjuntos de dados 20% de inadimplentes.

StandardScaler() - Foi escolhido para a padronização das variáveis porque teve o melhor desempenho em comparação com o MinMaxScaler() e o RobustScaler().

Ao avaliar o classificador obteve os seguintes resultados:

Acurácia: 0.99

A acurácia indica que 99% das observações foram corretamente classificadas, evidenciando o excelente desempenho global do modelo no conjunto de testes.

Precisão: 0.97

A precisão da classe positiva indica que, entre todos os contratos classificados como inadimplentes, 97% realmente apresentaram inadimplência, evidenciando baixo número de falsos positivos.

Recall: 0.96

O recall demonstra que 96% dos inadimplentes reais foram corretamente identificados, indicando alta capacidade de detecção da classe de risco.

F1 Score: 0.97

O F1-score sintetiza o equilíbrio entre precisão e recall. O valor de 0.97 demonstra que o modelo mantém simultaneamente baixa taxa de falsos positivos e baixa taxa de falsos negativos.

Segue o código do modelo:

PerceptronAccuracy.py

src > Perceptron > PerceptronAccuracy.py > ...

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Perceptron
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report

# Lendo os dados e criando o array com o nome das colunas.
data = pd.read_csv("data/mutuos_intercompany.csv")
yColumn = "default"

X = data.drop("default", axis=1)
y = data[yColumn]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, stratify=y, random_state=12)

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", Perceptron())
])

pipeline.fit(X_train, y_train)
y_pred = pipeline.predict(X_test)

print(classification_report(y_test, y_pred))
```

Segue o resultado do **classification_report**:

	precision	recall	f1-score	support
0	0.99	0.99	0.99	2400
1	0.97	0.96	0.97	600
accuracy			0.99	3000
macro avg	0.98	0.98	0.98	3000
weighted avg	0.99	0.99	0.99	3000

Apesar do alto desempenho observado, o Perceptron apresenta limitações estruturais:

- Incapacidade de modelar relações não lineares
- Sensibilidade a ruído e outliers

- Ausência de estimativa probabilística
- Dependência da separabilidade linear dos dados

Dessa forma, embora tenha apresentado excelente desempenho neste dataset, o modelo pode não generalizar adequadamente em cenários mais complexos.

O elevado desempenho sugere forte separabilidade linear entre as classes no conjunto de dados utilizado. Isso ocorre porque diversos indicadores financeiros utilizados possuem relação com o risco de crédito. Empresas com baixa liquidez, alta alavancagem, baixa cobertura de juros e histórico de atrasos tendem simultaneamente a apresentar maior probabilidade de inadimplência. Dessa forma, a combinação dessas variáveis é capaz de produzir um hiperplano separador relativamente claro entre contratos adimplentes e inadimplentes.

Ao analisar os coeficientes aprendidos de cada feature mostrados abaixo:

```
27   print(pipeline.named_steps["model"].coef_)
28   print(pipeline.named_steps["model"].intercept_)

PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

(venv) PS C:\Pedro\workspace\MutualCompliance> & C:/Pedro/workspace/MutualCompliance/venv/Scripts/python.exe c:/Pedro/workspace/MutualCompliance/src/Perceptron/Perceptron.py
              precision    recall  f1-score   support

           0       0.99      0.99      0.99      2400
           1       0.97      0.96      0.97       600

    accuracy                           0.99      3000
   macro avg       0.98      0.98      0.98      3000
weighted avg       0.99      0.99      0.99      3000

[[ -1.17373202   0.38259907   1.84748001 -17.08931968 -64.83152012
   41.69940355 -41.81662978  21.91058072 -12.42010337  -0.85887193
   23.00188296  17.53182371   0.81607978 -30.01493661 -28.41359704]]
 [-91.]
```

Concluímos que as features ebitda_margem, indice_liquidez_corrente, cobertura_juros, crescimento_receita_12m, rating_interno_score e garantia_ratio que possuem um valor negativo considerável indicam uma alta associação com um risco financeiro menor. Já as features indice_endividamento, volatilidade_receita, historico_atrasos_12m e exposicao_cambial tem um valor bem maior que 0 e indicam um risco financeiro maior.

Ou seja, o modelo conseguiu montar uma boa correlação entre as features para achar o perfil de cada classe, isso justifica o ótimo resultado encontrado.

O BIAS define a posição do hiperplano no espaço das features e para o modelo foi de -91, isto significa que ele tem uma forte tendência inicial para a classe 0 (adimplente) e que para um modelo classificar como inadimplente, a soma ponderada das variáveis precisa ser superior a -91 para o modelo prever inadimplência.

Modelo com Árvore de Decisão

Ao treinar um modelo simples sem nenhum hiperparâmetro, o modelo teve um resultado perfeito nos dados de treino e um resultado pouco melhor que o modelo Perceptron com os dados de teste. Porém, para estes resultados o modelo chegou a uma profundidade de 14 com o número de folhas 247. Embora para este conjunto de dados, o modelo tenha um resultado perfeito no treinamento e muito bom nos testes, este modelo provavelmente não se ajustará muito bem a novos dados, além de gerar um alto custo computacional.

Segue abaixo um print do código do modelo básico:

DecisionTree.py

src > DecisionTree > DecisionTree.py > ...

```python
import pandas as pd
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import classification_report

data = pd.read_csv("data/mutuos_intercompany.csv")
yColumn = "default"
RANDOM_STATE = 12

X = data.drop(yColumn, axis=1)
y = data[yColumn]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, stratify=y, random_state=RANDOM_STATE)

pipeline = Pipeline([
    ("model", DecisionTreeClassifier(random_state=RANDOM_STATE))
])

pipeline.fit(X_train, y_train)

yhat_train = pipeline.predict(X_train)
yhat_test = pipeline.predict(X_test)

print(classification_report(y_train, yhat_train))
print(classification_report(y_test, yhat_test))

# Analisando as dimensões da árvore criada
print(f'Profundidade da árvore: {pipeline.named_steps['model'].get_depth()}')
print(f'Número de folhas da árvore: {pipeline.named_steps['model'].get_n_leaves()}')
```

Segue a evidência dos resultados deste modelo:

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      5600
           1       1.00      1.00      1.00      1400

    accuracy                           1.00      7000
   macro avg       1.00      1.00      1.00      7000
weighted avg       1.00      1.00      1.00      7000

              precision    recall  f1-score   support

           0       0.99      0.99      0.99      2400
           1       0.97      0.98      0.97       600

    accuracy                           0.99      3000
   macro avg       0.98      0.99      0.98      3000
weighted avg       0.99      0.99      0.99      3000

Profundidade da árvore: 14
Número de folhas da árvore: 247
```

Validação cruzada e busca por hiperparâmetros

Estarei aplicando validação cruzada para todos os modelos seguintes para evitar generalização dos modelos, desta forma o modelo será treinado sobre vários subconjuntos de testes aleatórios e conseguirei encontrar o melhor modelo possível dentro do meu range de busca com o RandomSearch e GridSearch.

Para buscar a melhor combinação possível, irei primeiro executar um RandomizedSearch para visualizar as 20 melhores combinações de parâmetros dentre as que julgo que fazem sentido para a resolução do problema. Segue abaixo as combinações:

```
# configurar um espaço de busca
params_grid = {
    'dt__criterion': ['gini', 'entropy', 'log_loss'],
    'dt__max_depth': range(2, 10),
    'dt__splitter': ['best', 'random'],
    'dt__class_weight': [None, 'balanced'],
    'dt__min_samples_split': range(3, 10)
}
```

criterion - Estarei buscando modelos que utilizam diferentes métricas (gini, entropy ou log_loss) para determinar a melhor forma de dividir os nós da árvore;

max_depth - Estarei buscando modelos com profundidade máxima da árvore entre 2 e 9 níveis, controlando a complexidade do modelo e reduzindo risco de overfitting.

splitter - Estarei buscando modelos que utilizam diferentes estratégias para escolher os pontos de divisão da árvore, podendo selecionar a melhor divisão (best) ou introduzir aleatoriedade (random).

class_weight - Estarei buscando modelos que consideram ou não o balanceamento automático das classes, ajustando os pesos das observações quando houver desbalanceamento entre inadimplentes e não inadimplentes.

min_samples_split - Estarei buscando modelos onde um nó só pode ser dividido se possuir entre 3 e 9 amostras mínimas, controlando a granularidade das divisões da árvore.

Estarei avaliando os melhores modelos pela sua precisão e analisarei um total de **500** modelos.

Segue o resultado:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

[CV 2/4; 499/500] END dt__class_weight=balanced, dt__criterion=log_loss, dt__max_depth=3, dt__min_samples_split=9, dt__splitter=random;, score=0.460 total time=   0.0s
[CV 3/4; 499/500] START dt__class_weight=balanced, dt__criterion=log_loss, dt__max_depth=3, dt__min_samples_split=9, dt__splitter=random
[CV 3/4; 499/500] END dt__class_weight=balanced, dt__criterion=log_loss, dt__max_depth=3, dt__min_samples_split=9, dt__splitter=random;, score=0.470 total time=   0.0s
[CV 4/4; 499/500] START dt__class_weight=balanced, dt__criterion=log_loss, dt__max_depth=3, dt__min_samples_split=9, dt__splitter=random
[CV 4/4; 499/500] END dt__class_weight=balanced, dt__criterion=log_loss, dt__max_depth=3, dt__min_samples_split=9, dt__splitter=random;, score=0.433 total time=   0.0s
[CV 1/4; 500/500] START dt__class_weight=balanced, dt__criterion=gini, dt__max_depth=9, dt__min_samples_split=3, dt__splitter=best
[CV 1/4; 500/500] END dt__class_weight=balanced, dt__criterion=gini, dt__max_depth=9, dt__min_samples_split=3, dt__splitter=best;, score=0.788 total time=   0.0s
[CV 2/4; 500/500] START dt__class_weight=balanced, dt__criterion=gini, dt__max_depth=9, dt__min_samples_split=3, dt__splitter=best
[CV 2/4; 500/500] END dt__class_weight=balanced, dt__criterion=gini, dt__max_depth=9, dt__min_samples_split=3, dt__splitter=best;, score=0.809 total time=   0.0s
[CV 3/4; 500/500] START dt__class_weight=balanced, dt__criterion=gini, dt__max_depth=9, dt__min_samples_split=3, dt__splitter=best
[CV 3/4; 500/500] END dt__class_weight=balanced, dt__criterion=gini, dt__max_depth=9, dt__min_samples_split=3, dt__splitter=best;, score=0.759 total time=   0.0s
[CV 4/4; 500/500] START dt__class_weight=balanced, dt__criterion=gini, dt__max_depth=9, dt__min_samples_split=3, dt__splitter=best
[CV 4/4; 500/500] END dt__class_weight=balanced, dt__criterion=gini, dt__max_depth=9, dt__min_samples_split=3, dt__splitter=best;, score=0.809 total time=   0.0s
{'dt__splitter': 'best', 'dt__min_samples_split': 5, 'dt__max_depth': 9, 'dt__criterion': 'entropy', 'dt__class_weight': None}
0.9023855291365701
```

O modelo com melhor precisão possui os parâmetros “splitter=best”, “min_samples_split=5”, “max_depth=9”, “criterion=entropy” e “class_weight=None”. A sua precisão foi de 90,23%.

Agora se analisarmos o desempenho do melhor modelo encontrado treinado com todos os dados e testado com os dados previamente separados, teremos os seguintes resultados:

```
Desempenho - Base de Treino
              precision    recall  f1-score   support

           0       0.99      0.99      0.99      5600
           1       0.95      0.95      0.95      1400

    accuracy                           0.98      7000
   macro avg       0.97      0.97      0.97      7000
weighted avg       0.98      0.98      0.98      7000

Desempenho - Base de Teste
              precision    recall  f1-score   support

           0       0.98      0.97      0.97      2400
           1       0.88      0.90      0.89       600

    accuracy                           0.96      3000
   macro avg       0.93      0.94      0.93      3000
weighted avg       0.96      0.96      0.96      3000
```

Isso significa que o melhor modelo teve um resultado excelente com os dados de treino e teste sem que a árvore escale até a profundidade máxima e gerando um custo computacional muito menor.

Contudo, o RandomizedSearch gera combinações de modelos aleatoriamente, logo, não necessariamente este é o melhor modelo possível.

Para encontrar o melhor modelo possível usando a árvore de decisão irei encontrar os 20 melhores modelos e utilizar o GridSearch para limitar a minha busca pelo melhor modelo possível de custo x benefício usando a árvore de decisão.

Segue abaixo a lista dos 20 melhores modelos encontrados:

	dt__splitter	dt__min_samples_split	dt__max_depth	dt__criterion	dt__class_weight
153	best	5	9	entropy	NaN
285	best	5	9	log_loss	NaN
325	best	4	9	entropy	NaN
39	best	4	9	log_loss	NaN
32	best	3	9	gini	NaN
352	best	3	9	log_loss	NaN
141	best	3	9	entropy	NaN
211	best	6	9	entropy	NaN
160	best	7	9	entropy	NaN
351	best	7	9	log_loss	NaN
141	best	3	9	entropy	NaN
211	best	6	9	entropy	NaN
160	best	7	9	entropy	NaN
141	best	3	9	entropy	NaN
211	best	6	9	entropy	NaN
141	best	3	9	entropy	NaN
141	best	3	9	entropy	NaN
211	best	6	9	entropy	NaN
160	best	7	9	entropy	NaN
351	best	7	9	log_loss	NaN
257	best	8	9	log_loss	NaN
393	best	8	9	entropy	NaN
47	best	5	9	gini	NaN
72	best	9	9	log_loss	NaN
265	best	9	9	entropy	NaN
409	best	7	9	gini	NaN
278	best	4	8	log_loss	NaN
290	best	5	8	entropy	NaN
482	best	5	8	log_loss	NaN
36	best	3	8	log_loss	NaN

Ao analisarmos os melhores modelos podemos concluir que os parâmetros a serem utilizados no GridSearch são os seguintes:

splitter = best;

min_samples_split = varia muito, porém principalmente entre um range de **3 a 7**, logo podemos diminuir a busca por esse parâmetro;

max_depth = 9;

criterion = Varia bastante, porém no Top 10 só tem um Gini e no Top 20 apenas 3 ao total, logo, podemos concluir que estatisticamente este parâmetro tende a não ser o melhor.

class_weight = None;

Com isso, segue abaixo os parâmetros que utilizarei no meu GridSearch para encontrar o melhor modelo possível de árvore de decisão:

```python
# configurar um espaço de busca
params_grid = {
    'dt__criterion': ['entropy', 'log_loss'],
    'dt__max_depth': [9],
    'dt__splitter': ['best'],
    'dt__class_weight': [None],
    'dt__min_samples_split': range(3, 7)
}
```

Como resultado, obtive o modelo abaixo:

```
{'dt__class_weight': None, 'dt__criterion': 'entropy', 'dt__max_depth': 9, 'dt__min_samples_split': 5, 'dt__splitter': 'best'}
0.9023855291365701
Desempenho - Base de Treino
              precision    recall  f1-score   support

           0       0.99      0.99      0.99      5600
           1       0.95      0.95      0.95      1400

    accuracy                           0.98      7000
   macro avg       0.97      0.97      0.97      7000
weighted avg       0.98      0.98      0.98      7000

Desempenho - Base de Teste
              precision    recall  f1-score   support

           0       0.98      0.97      0.97      2400
           1       0.88      0.90      0.89       600

    accuracy                           0.96      3000
   macro avg       0.93      0.94      0.93      3000
weighted avg       0.96      0.96      0.96      3000
```

Por sorte, este modelo é o mesmo encontrado no RandomizedSearch.

Por fim, ao analisarmos o modelo de árvores de decisão puro, com hiperparâmetros e o modelo Perceptron, concluo que o melhor modelo permanece sendo o Linear devido a resultados melhores que o de árvore de decisão com hiperparâmetros e um custo menor que o puro, ou seja, ele ainda tem um melhor custo x benefício.

Modelo com Support Vector Machine (SVM)

Primeiramente irei utilizar o RandomSearch para treinar diversos modelos de SVM com validação cruzada. Segue abaixo os parâmetros que irei utilizar:

```
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("svc", SVC(random_state=RANDOM_STATE))
])

# configurar um espaço de busca
params_grid = {
    'svc__kernel': ['linear', 'rbf', 'sigmoid', 'poly'],
    'svc__C': np.random.uniform(0.01, 100, 200),
    'svc__class_weight': [None, 'balanced']
}

# configurar um amostrador estratificado
splitter = StratifiedKFold(n_splits=4, shuffle=True, random_state=RANDOM_STATE)

# configurar o nosso "experimentador"
random_search = RandomizedSearchCV(
    estimator=pipeline,
    param_distributions=params_grid,
    scoring='precision',
    n_iter=500,
    cv=splitter,
    refit=True,
    verbose=10,
    error_score=0,
    random_state=RANDOM_STATE
)
```

Estou utilizando o StandarScaler na minha pipeline porque ao avaliar o modelo Perceptron identifiquei que este Scaler se adequa melhor aos meus dados.

Nos parâmetros do meu espaço de busca estou usando as quatro opções existentes para o hiperparâmetro “**kernel**”. Além disso, para o meu parâmetro “C” que é o controle de regularização do modelo SVM estou usando **200** parâmetros aleatórios entre **0.01 e 100**. Para o hiperparâmetro class_weight utilizei os mesmos parâmetros do meu modelo com árvore de decisão.

Dentro de um total de **1600** modelos, treinei **500** com validação cruzada. Segue abaixo o modelo com a maior precisão encontrada, a precisão dele e o desempenho dele tanto na base total de treinamento quanto na de testes:

```
{'svc__kernel': 'linear', 'svc__class_weight': None, 'svc__C': np.float64(87.48635893714844)}
0.9985795454545454
Desempenho - Base de Treino
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      5600
           1       1.00      1.00      1.00      1400

    accuracy                           1.00      7000
   macro avg       1.00      1.00      1.00      7000
weighted avg       1.00      1.00      1.00      7000

Desempenho - Base de Teste
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      2400
           1       1.00      0.99      1.00       600

    accuracy                           1.00      3000
   macro avg       1.00      1.00      1.00      3000
weighted avg       1.00      1.00      1.00      3000
```

Como podemos analisar na imagem acima, este modelo teve um desempenho quase perfeito para a base de treinamento e de testes, porém com um controle de regularização bem alto, ou seja, ele penaliza os erros mais severamente e prioriza uma margem mais estreita.

Se analisarmos o top 50 melhores modelos da imagem abaixo, todos são modelos com **kernel = "linear"** e com **class_weight = None**. Olhando para o hiperparâmetro "**C**", no top 25 temos valores entre **52 e 87,50**.

Embora tenhamos encontrado um modelo quase perfeito para o conjunto de dados, iremos em busca do melhor custo x benefício utilizando o GridSearch.

	svc__kernel	svc__class_weight	svc__C
453	linear	NaN	85.709246
38	linear	NaN	87.486359
368	linear	NaN	86.100624
124	linear	NaN	84.865864
397	linear	NaN	65.775109
73	linear	NaN	69.442908
96	linear	NaN	62.530054
425	linear	NaN	66.183238
105	linear	NaN	71.996741
322	linear	NaN	58.458277
146	linear	NaN	60.915469
227	linear	NaN	71.152738
199	linear	NaN	60.178862
8	linear	NaN	64.258606
172	linear	NaN	64.032282
351	linear	NaN	82.504705
188	linear	NaN	79.710822
16	linear	NaN	53.014424
184	linear	NaN	52.684068
187	linear	NaN	52.696508
176	linear	NaN	56.422757
88	linear	NaN	56.160806
430	linear	NaN	58.008657
444	linear	NaN	57.375245
93	linear	NaN	43.886711
110	linear	NaN	41.705727
110	linear	NaN	41.705727
296	linear	NaN	38.403041
115	linear	NaN	42.569607
255	linear	NaN	41.501617
251	linear	NaN	50.338920
251	linear	NaN	50.338920
138	linear	NaN	47.842629
145	linear	NaN	35.992371
499	linear	NaN	47.926603
148	linear	NaN	42.114847
203	linear	NaN	36.914603
437	linear	NaN	33.323912
441	linear	NaN	34.162073
306	linear	NaN	34.000714
383	linear	NaN	9.171162
490	linear	NaN	97.608157
23	linear	NaN	14.849983
216	linear	NaN	28.550895
104	linear	NaN	29.655372
159	linear	NaN	25.419433
497	linear	NaN	12.330269
230	linear	NaN	12.639488
57	linear	NaN	14.480084
0	linear	NaN	22.417144
398	linear	NaN	18.421233
232	linear	NaN	5.541811

Utilizando o GridSearch no meu SVM Linear

Após a análise inicial utilizando o RandomSearch, eu irei utilizar os hiperparâmetros **kernel = “linear”**, **class_weight = “None”** e “**C**” = entre **52 e 87,50**.

Como já encontramos alguns parâmetros ideais para os hiperparâmetros “**kernel**” e “**class_weight**”, eu irei manter a quantidade de “**C**” para **200**. Desta forma treinarei **200** modelos com validação cruzada.

Segue abaixo o trecho de código:

```
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("svc", SVC(random_state=RANDOM_STATE))
])

# configurar um espaço de busca
params_grid = {
    'svc__kernel': ['linear'],
    'svc__C': np.random.uniform(52, 87.5, 200),
    'svc__class_weight': [None]
}

# configurar um amostrador estratificado
splitter = StratifiedKFold(n_splits=4, shuffle=True, random_state=RANDOM_STATE)

# configurar o nosso "experimentador"
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=params_grid,
    scoring='precision',
    cv=splitter,
    refit=True,
    verbose=10,
    error_score=0
)
```

Como resultado obtive o modelo abaixo com a mesma precisão média da validação cruzada e um resultado quase perfeito, porém com um “C” diferente do modelo encontrado no RandomSearch.

```
{'svc__C': np.float64(85.04942140684679), 'svc__class_weight': None, 'svc__kernel': 'linear'}
0.9985795454545454
Desempenho - Base de Treino
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      5600
           1       1.00      1.00      1.00      1400



    accuracy                           1.00      7000
   macro avg       1.00      1.00      1.00      7000
weighted avg       1.00      1.00      1.00      7000

Desempenho - Base de Teste
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      2400
           1       1.00      0.99      1.00       600

    accuracy                           1.00      3000
   macro avg       1.00      1.00      1.00      3000
weighted avg       1.00      1.00      1.00      3000
```

Se compararmos o modelo SVM com os modelos anteriores, este teve o melhor resultado considerando a base inteira de treinamento e teste.

Modelos Ensemble - Gradient Boosting (XGB)

Eu estarei criando um modelo apenas com Gradient Boosting porque ao compararmos especialmente com o RandomForest, o XGB possui uma alta sensibilidade a outliers, ou seja, pode focar demais nos ruídos, porém analisando os resultados dos modelos anteriores com o meu conjunto de dados, acredito que os meus dados possui pouco ruído. Pelo XGB possuir um processamento sequencial que vai considerando os erros de cada modelo anterior, ele pode me entregar um resultado melhor que o Random Forest. Por estes motivos estarei testando apenas o Gradient Boosting.

Para encontrar o melhor modelo possível de XGB, irei usar o mesmo passo a passo da árvore de decisão e SVMs. Procurarei a melhor possibilidade de parâmetros com o RandomSearch para limitar a minha busca com o GridSearch, então chegarei no que tende a ser o melhor modelo possível.

Os parâmetros escolhidos para o meu RandomSearch foram:

k (SelectorKBest) = 5, 10 ou todas (15). Como o meu conjunto de dados possui um total de 15 features, acredito que esta divisão será suficiente para encontrar um bom modelo;

n_estimators = 40, 120, 190 ou 400. Este parâmetro define quantas árvores o modelo irá construir, não sei se tem a ver mas como o meu modelo de árvore de decisão sem limite de profundidade chegou a 247 folhas, resolvi procurar um modelo com essa probabilidade de quantidade de árvores;

learning_rate = 500 valores aleatórios de 0,001 a 10. Considerando que quanto menor o número mais lento será o aprendizado do próximo modelo e quanto maior mais rápido ele será, julgo que está será uma boa variação de parâmetros;

Como aplicarei validação cruzada nessa combinação de parâmetros, escolherei aleatoriamente **300** modelos para ter uma noção mais aproximada dos melhores parâmetros para então aplicar o meu GridSearch.

Segue abaixo um print dos parâmetros do meu RandomSearch:

```python
pipeline = Pipeline([
    ('selector', SelectKBest(f_classif)),
    ("scaler", StandardScaler()),
    ("xgb", XGBClassifier(random_state=RANDOM_STATE))
])

# configurar um espaço de busca
params_grid = {
    'selector__k': [5, 10, X_train.shape[1]],
    'xgb__n_estimators': [40, 120, 190, 400],
    'xgb__learning_rate': np.random.uniform(0.001, 10, 500)
}

# configurar um amostrador estratificado
splitter = StratifiedKFold(n_splits=4, shuffle=True, random_state=RANDOM_STATE)

# configurar o nosso "experimentador"
random_search = RandomizedSearchCV(
    estimator=pipeline,
    param_distributions=params_grid,
    scoring='precision',
    n_iter=300,
    cv=splitter,
    refit=True,
    verbose=10,
    error_score=0,
    random_state=RANDOM_STATE
)
```

Como resultado a precisão média do melhor modelo foi 98,239119%. Se analisarmos o desempenho do modelo na base inteira de treinamento e de testes, obtive o mesmo resultado do meu modelo SVM. Segue abaixo a evidência dos resultados:

```
{'xgb__n_estimators': 190, 'xgb__learning_rate': np.float64(0.40455097430610526), 'selector__k': 10}
0.9823911908420798
Desempenho - Base de Treino
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      5600
           1       1.00      1.00      1.00      1400

    accuracy                           1.00      7000
   macro avg       1.00      1.00      1.00      7000
weighted avg       1.00      1.00      1.00      7000

Desempenho - Base de Teste
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      2400
           1       1.00      0.99      1.00       600

    accuracy                           1.00      3000
   macro avg       1.00      1.00      1.00      3000
weighted avg       1.00      1.00      1.00      3000
```

Se olharmos agora para os 30 melhores modelos encontrados no RandomSearch do imagem abaixo:

	xgb__n_estimators	xgb__learning_rate	selector__k
234	190.0	0.404551	10.0
134	190.0	0.230462	10.0
188	190.0	0.462297	15.0
17	400.0	0.773132	10.0
265	400.0	0.230462	15.0
210	120.0	0.514249	15.0
83	190.0	1.030838	10.0
137	120.0	0.042134	10.0
109	190.0	1.030838	15.0
288	120.0	0.669465	10.0
218	120.0	0.462297	15.0
130	190.0	0.669465	15.0
168	190.0	0.812932	15.0
39	40.0	0.719587	10.0
279	400.0	0.669834	10.0
92	120.0	0.842418	10.0
264	40.0	0.042134	10.0
51	120.0	1.344281	10.0
289	40.0	0.286319	10.0
173	190.0	0.966038	15.0
129	40.0	1.235480	10.0
94	190.0	1.568359	10.0
5	40.0	0.966038	10.0
235	400.0	1.493784	15.0
29	40.0	1.484934	15.0
277	190.0	1.568359	15.0
120	40.0	1.493784	15.0
226	400.0	1.483802	15.0
159	120.0	1.674149	15.0
261	190.0	1.764448	15.0

Vemos que a quantidade de features utilizados pelo modelo varia entre **10 e 15**, isso faz sentido porque por regra de negócio, para traçar o perfil de uma empresa é preciso analisar vários dados em conjunto.

A taxa de aprendizagem dos modelos variou bastante entre **0.23 e 1.03** nos 10 melhores modelos, logo, já temos uma estimativa mais aproximada de qual valor buscar.

A quantidade de árvores do modelo variou bastante, principalmente entre **190 e 120**, logo, eliminaremos os valores de 40 e 400 do meu GridSearch.

Baseado na minha análise anterior do meu modelo XGB com RandomSearch, irei utilizar os parâmetros abaixo para a busca do meu modelo:

```python
pipeline = Pipeline([
    ('selector', SelectKBest(f_classif)),
    ("scaler", StandardScaler()),
    ("xgb", XGBClassifier(random_state=RANDOM_STATE))
])

# configurar um espaço de busca
params_grid = {
    'selector__k': [10, 15],
    'xgb__n_estimators': [120, 190],
    'xgb__learning_rate': np.random.uniform(0.23, 1.03, 150)
}

# configurar um amostrador estratificado
splitter = StratifiedKFold(n_splits=4, shuffle=True, random_state=RANDOM_STATE)

# configurar o nosso "experimentador"
grid_search = GridSearchCV(
    estimator=pipeline,
    param_grid=params_grid,
    scoring='precision',
    cv=splitter,
    refit=True,
    verbose=10,
    error_score=0
)
```

Eu obtive o seguinte resultado:

```
{'selector__k': 10, 'xgb__learning_rate': np.float64(0.5684730220681655), 'xgb__n_estimators': 120}
0.9870071724217209
Desempenho - Base de Treino
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      5600
           1       1.00      1.00      1.00      1400

    accuracy                           1.00      7000
   macro avg       1.00      1.00      1.00      7000
   macro avg       1.00      1.00      1.00      7000
weighted avg       1.00      1.00      1.00      7000

Desempenho - Base de Teste
weighted avg       1.00      1.00      1.00      7000

Desempenho - Base de Teste

Desempenho - Base de Teste
Desempenho - Base de Teste
              precision    recall  f1-score   support
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      2400
           1       0.99      0.99      0.99       600

    accuracy                           1.00      3000
   macro avg       0.99      0.99      0.99      3000
weighted avg       1.00      1.00      1.00      3000
```

Comparando o desempenho médio deste modelo com o anterior, obtive um resultado melhor, porém quando vou treinar e testar o modelo com todos os dados, na minha base de teste recebi um resultado levemente pior que o modelo anterior encontrado no meu RandomSearch.

Justificativas

Porque avaliei todos os modelos pela precisão ?

A precisão da classe positiva é mais importante do que a classe negativa porque se eu bloqueio um contrato de mútuo quando não deveria, normalmente os impactos operacionais e financeiros acabam sendo maiores do que a inadimplência de um contrato. Por outro lado, a imprecisão poderia gerar os problemas mencionados na descrição do problema atual.

Porque utilizei na validação cruzada uma divisão por 4 na minha base de treinamento ?

Porque como tenho um total de 10 mil linhas no meu conjunto de dados sendo que 3 mil eu separei para os meus testes, logo, com 7 mil linhas, se eu dividir em muitas partes teria muitos poucos dados para o treinamento e diminuiria a confiabilidade na qualidade do modelo.

Comparando os modelos e entendendo os resultados encontrados

O primeiro ponto e mais importante é sobre a qualidade dos dados. Os dados possuem pouco ruído, porém para identificar os padrões é necessário uma certa complexidade, por isso todos os modelos tiveram um resultado muito bom. A maior prova que os dados possuem pouco ruído é que quando treinei o modelo de árvore de decisão sem limitar a profundidade da árvore e fui testar o modelo obtive um resultado muito bom, ou seja, os dados se encaixaram muito bem em todas as condições da árvore.

Ao compararmos todos os resultados, o pior modelo foi o de árvore de decisão, principalmente o modelo com os melhores hiperparâmetros encontrados. O modelo de árvore de decisão sem limitação de profundidade obteve um resultado muito próximo do nosso modelo baseline, porém com um custo computacional maior, logo, concluo que até o momento o melhor modelo ainda é o nosso baseline (Perceptron).

Ao analisarmos os modelos SVM e XGB (Ensemble), ambos tiveram um resultado quase perfeito para os dados de treinamento e teste, porém ambos levaram consideravelmente mais tempo para o treino e geraram um custo computacional maior, especialmente o XGB.

Considerando o custo x benefício de cada modelo e o impacto de um erro na operação de uma holding que normalmente administra os recursos de várias empresas do mesmo grupo, o modelo escolhido para solucionar o problema é o Support Vector Machine (SVM).

Como validação final, criei o modelo do arquivo ConclusionModel.py com base no meu modelo SVM e testei o modelo com dois empréstimos de duas empresas, um da petrobras e outro dos correios. Nem todos os dados encontrei para fazer um teste 100% fidedigno, porém baseado no momento e dados financeiros das duas empresas acreditamos que os Correios têm uma tendência muito maior a inadimplência.

Como resultado da minha validação final, obtive o resultado abaixo sendo previsto que tanto o empréstimo mais recente para a Petrobrás de 3 bilhões e o dos Correios de 8 bilhões o modelo previu como inadimplência.

```
Previsões: [1 1]
```

Se alterar o empréstimo da Petrobrás para 300 milhões, o modelo prevê como adimplência. Logo, concluo que o modelo está mal treinado para valores maiores e que os dados não são bons.

Você pode ter se perguntado, se o modelo é para resolver um problema de mútuo entre empresas, porque estou testando com um empréstimo bilhonário que não é real ?

Por regra de negócio, um contrato de mútuo nada mais é que um contrato de empréstimo mas entre empresas do mesmo grupo e não com um banco ou qualquer outra instituição financeira e é pouco comum um empréstimo de valores tão grandes mesmo em cenários de grupos gigantescos que possuem números muitos grandes, logo, posso considerar que o modelo não é inútil, porém mesmo com um resultado quase perfeito nos testes ele ainda pode prever errado alguns contratos. Isso indica que ainda podemos melhorar nosso modelo treinando ele com mútuos maiores ou alterar a escala de alguns dados e refazendo o processo de experimentação e treinamento para encontrar um modelo que se adeque a diversos cenários e mútuos e não apenas os mais comuns.